

Mini-Lab: Secure-Code Audit (Take-home)

Vulnerable Microblog (Flask + SQLite)

ECE574 / SENG460 — Practice of Information Security & Privacy

Estimated Time: **at most 6 hours per group**

Overview

You are given a small Flask + SQLite web application that is **intentionally vulnerable**. Your task is to **audit**, **patch**, and **lock-in fixes with regression tests**.

This lab emphasizes security as **invariant preservation** across trust boundaries: untrusted input, authorization decisions, state-changing actions, error handling, and file access.

Learning Objectives

By the end of this lab, you should be able to:

- Identify implementation bugs that violate **security invariants**.
- Patch common vulnerabilities: **SQL injection**, **broken access control (IDOR)**, **weak session handling**, **CSRF**, **information leakage**, **sensitive logging**, **unsafe file/path handling**, and **weak password storage**.
- Write **regression tests** (minimum 4) that prevent reintroducing vulnerabilities.
- Explain fixes using **trust boundaries** and **invariants**.

Provided Starter Code

You received a ZIP repository (“secure_audit_lab”) containing:

- `app.py`, `db.py`, `auth.py`
- `templates/` (server-rendered pages)
- `tests/` (pytest skeleton)
- `README.md` with setup instructions

Setup (Quick)

1. Create a virtual environment and install dependencies:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

2. Run the app:

```
python app.py
```

3. Open: `http://127.0.0.1:5000`

Your Tasks

Task 1: Explore the App (20–30 min)

- Create two users (e.g., Alice and Bob).
- Create posts (including a private post).
- Try editing/deleting posts and viewing private content.

Task 2: Audit (60–90 min)

Find and document at least **6 vulnerabilities** (there are **8** in the codebase). For each vulnerability, record:

- **Where it is** (file + function/route)
- **What the impact is** (what breaks, what attacker gains)
- **Which invariant is violated**

Task 3: Patch (60–90 min)

Implement fixes. Your patch quality matters: favor **clear, minimal, correct** changes.

Minimum expected fixes include:

- Replace SQL string concatenation with **parameterized queries**.
- Enforce **authorization checks** for edit/delete and private post viewing.
- Replace forgeable identity cookie with **integrity-protected session**.
- Add **CSRF protection** for state-changing POST routes.
- Eliminate **credential logging** and reduce information leakage in errors.
- Prevent path traversal in export (enforce directory containment).
- Store passwords using **strong one-way hashing**.

Task 4: Add Tests (30–45 min)

Add at least **4 meaningful regression tests** in `tests/`. Your tests should demonstrate that your security fixes hold.

Recommended tests:

- SQL injection attempt should **not** bypass login/search.
- Alice **cannot** delete/edit Bob's post.
- POST without CSRF token is **rejected**.
- Export output path cannot escape the allowed **data/** directory.

Run tests:

```
pytest -q
```

Student Checklist: Security Invariants to Enforce

Use this checklist to guide your audit and patching:

- Untrusted input must not alter SQL structure.
- Users can modify only resources they own (unless admin).
- Identity tokens must be unforgeable and integrity-protected.
- State-changing requests must verify user intent (CSRF).
- Private data must not be disclosed to unauthorized users.
- Secrets must never be logged.
- Passwords must be stored using strong one-way hashing.
- File paths from user input must stay within allowed directories.
- Debug/verbose errors must not be exposed in production.
- Security checks must be atomic with protected actions.

Deliverables

Submit a single ZIP (or repo link) containing:

1. **Patched source code**
2. **Tests** (at least 4 regression tests)
3. **Audit report** (1–2 pages max; PDF or Markdown)

Audit report format (recommended table):

- Vulnerability
- Exploit idea (high level; do not provide weaponized steps)
- Impact
- Fix summary
- Invariant enforced
- Test added (name + what it asserts)

Grading (100 pts + bonus)

Category	Points
Vulnerability identification (6 correctly explained)	30
Fix correctness & security quality	40
Regression tests (4 meaningful tests)	20
Audit report clarity (bug → impact → fix → invariant)	10
Bonus (hardening: rate-limits, headers, improved logging, etc.)	+10

Policy: AI Assistance (Allowed with Disclosure)

You may use generative AI tools as a **learning aid**, for example to:

- explain concepts (SQL injection, CSRF, sessions, access control),
- suggest safer library APIs (parameterized queries, hashing),
- help write **unit tests** and improve code readability.

You may **not** use AI to:

- paste the entire codebase and request a full end-to-end solution,
- generate a complete patch set without your own reasoning.

Disclosure requirement: If you used AI assistance, include a short note (2–4 lines) in your report:

Tool used, what it helped with (e.g., “suggested CSRF token pattern”), and what you implemented yourself.

Academic Integrity Note

This is an **audit** exercise. Your explanation and tests must reflect your own understanding. Copying fixes from other students or public solutions is not permitted.

Submission

Submit via the course LMS (instructions will be provided). Ensure:

- Code runs, tests pass (`pytest -q`).
- Report is included and readable.
- You did not commit secrets or credentials.